

CSC4026Z Chat Server Documentation

Chat protocol

The Chat protocol consists of individual messages sent either by the client or the server. Because we rely on UDP, there is an upper bound on the size of any individual message, approximately 1460 bytes depending on your local network conditions.

Encoding

All messages between the Client and Server **MUST** be encoded with [MessagePack](#), which is a binary serialisation format:

```
>>> import msgpack
>>> packed = msgpack.packb({'i': 'am', 'a': ['complex', 'python', ('ob
>>> packed
b'\x82\xa1i\xa2am\xa1a\x93\xa7complex\xa6python\x91\xa6object'

>>> msgpack.unpackb(packed)
{'i': 'am', 'a': ['complex', 'python', ['object']]}
```

Every message is a Python dictionary with fields specified in the message descriptions. To give you a small taste, the [CONNECT](#) message is simply:

```
connect_data = {'request_type': 1, 'request_handle': random.randint(0,
packet_data = msgpack.packb(connect_data)
```

The server will return an error if the packet contents are not valid MessagePack.

Session Management

Because UDP is connectionless, we rely on a `session` field to tie packets of the same session together. Every message (except the [CONNECT](#) request) **MUST** include the `session` field. It stays constant for your chat session, but will change the next time you connect. The server is responsible for generating the session identifier, and returns it as part of the [CONNECT](#) response.

Request / Response

Most messages are triggered by some user action, such as joining a channel, sending a message to a channel or user, looking up info on a channel, changing your username, and so on. The client sends a request, and when the response arrives from the server it knows the server processed the request.

Your client may send several requests quickly; in order to tie a request to the server's response, every request **MUST** include a `request_handle` field. The `request_handle` is a random positive integer less than 2^{32} . You can generate it with:

```
>>> import random
>>> request_handle = random.randrange(0, 2**32)
```

The response corresponding to your request will include the same handle, but in the `response_handle` field.

Messages from the server **MAY** include data in the `response_handle` field. If you receive a message with an empty `response_handle` field, it means you received a message you didn't request. This is expected; actions like other users sending messages in channels or as DMs, or joining/leaving channels, were not as a result of your request, but you still are kept informed.

The rule of thumb is: if the response contains a non-empty `response_handle` field then it's because you asked the server for something. If `response_handle` is empty, someone else asked the server to do something and the server wants to tell you about it.

Idle sessions

The Chat Server will delete idle session. An idle session is one where no messages have been seen from the client in 60 seconds. Get around this by sending a [PING](#) request every 30 seconds, which keeps your session alive.

Message flow

1. You **MUST** always start with a [CONNECT](#) message to obtain a valid session identifier.
2. You **MAY** set your username at any stage, but doing it a

connect time makes sense.

3. You **MAY** join and leave channels, pull the user list, send messages and so on.
4. You **MUST** continue to send a [PING](#) message every 30 seconds, otherwise your session will expire.
5. When you're done, you **MAY** send a [DISCONNECT](#). If you don't, the server will simply expire your session anyway if it hasn't received messages from you.

Message formats

These are documented in the Message Types item on the menu bar, you can start with [Session handling](#).

Documentation built with [MkDocs](#).